

s1.01 — Distributed rate limiter with per-tenant quotas

Index

- [Interview framing](#)
- [Requirements — IC3 baseline](#)
- [Capacity and consequence — IC3 baseline](#)
- [Architecture and flow — IC3 baseline](#)
- [API and data model — IC3 baseline](#)
- [Atomic algorithm and races — IC3 baseline](#)
- [Failure, operations, and security](#)
- [Alternatives and IC4 stretch](#)
- [Level signal — IC3 baseline vs IC4 stretch](#)
- [Interview pushback and validation](#)
- [Recall card](#)
- [References](#)

Interview framing

- Target: OCI IC3 ownership of a bounded platform component.
 - `IC3 baseline` is what must land in every phase.
 - `IC4 stretch` adds architecture changes under stronger constraints.
 - These are preparation labels, not Oracle's official rubric.
- Evidence: retain the first-hand report attribution in [the bank](#).
 - The architecture below is a proposed interview answer.
 - It does not claim to reproduce Oracle's implementation or an interviewer's accepted answer.
- Why this is hard
 - Naive: a counter per tenant in each gateway's memory, reset every second.
 - First break: `N` gateways each hold a full budget, so a tenant gets `Nx` the quota, and a restart wipes the count.
 - Second break: the shared owner times out after debiting, or fails over with a lagging replica, and nobody knows whether the tokens were spent.
 - Minimum primitives
 - One atomic refill-and-debit per bucket
 - A stable bucket key
 - A versioned policy snapshot
 - An explicit degradation rule per policy.
- 🗣️: "The algorithm is ten lines; the design is who owns the bucket and what happens when that owner cannot answer in time."
- Opening script
 - "I will design a regional per-tenant token bucket with one atomic owner per bucket, then make its timeout and failover behavior explicit."
 - "First I confirm whether bursts are allowed, whether the limit is regional or global, and whether it protects capacity or a security boundary."
 - "Then I size bucket state and decision traffic, and define an internal check API plus an admin policy API."
 - "The deep dive is the atomic refill-and-consume script, its clock handling, and the ambiguous-timeout race."
 - "I will close with the failure policy per rule, the hot-tenant limit, and what changes under a global or exact budget."

Phase	Clock	Deliverable
Intro and behavioral slice	00:00–05:00	90-second pitch; confirm the prompt
Requirements	05:00–11:00	Scope, clarifying answers, burst-envelope invariant
Estimates and contract	11:00–17:00	Bucket state and shard count; check and policy API semantics
Architecture	17:00–30:00	Diagram, control/data plane split, one decision trace
Deep dive	30:00–45:00	Atomic script, clock, ambiguous timeout, multi-bucket debit
Operations and tradeoffs	45:00–50:00	Failure table, degradation policy, tradeoff, wrap script
Wrap-up	50:00–54:00	Four-sentence recap and weakest assumption
Questions for interviewer	54:00–60:00	Two role-specific questions

- Clock note
 - If the interviewer states 45 minutes, drop the intro to 2 minutes and compress each phase proportionally.
 - The interviewer's clock wins over this table.

Requirements — IC3 baseline

- Numerical targets are interview assumptions unless an adjacent primary source states otherwise.

Question	Assumed answer	Why it changes the design
Are bursts allowed?	Yes, up to a configured burst <code>B</code>	Selects token bucket over a sliding log or fixed window
Regional or global limit?	Regional; global is a stretch	One regional owner per bucket avoids cross-region coordination on every request
Capacity protection or security boundary?	Both exist; each rule declares which	Decides fail-open with a bound versus fail-closed on owner loss
What identifies the quota?	Authenticated tenant plus normalized route class	Attacker-controlled raw URLs cannot mint new buckets
Is an exact rolling-window count required?	No; a burst envelope is acceptable	Exactness would force a sliding log and stronger state ownership
Who can change limits?	Tenant administrators through a versioned policy API	Policy becomes a control-plane resource with audit and rollout

- Functional scope
 - Decide allow or reject for `(tenant, route class, cost)` before expensive backend work.
 - Permit a configured burst and refill at a configured rate.
 - Serialize each bucket's decisions at one regional owner.
 - Distribute versioned policies to gateways without a restart.
 - Return `429` with a retry hint
 - Report degraded admission separately from normal success.

Requirement	Target	Enforced by
Regional decision throughput	<code>100,000</code> checks/s	Sharded bucket owners; one atomic operation per check
Decision latency	<code>p99 < 2 ms</code> in-region	Single round trip to a colocated owner; no cross-region call on the request path
Decision timeout	Stop waiting at <code>5 ms</code>	Gateway deadline; then the rule's explicit degradation policy
Decision-service availability	<code>99.95%</code>	Replicated owner per shard; gateways keep a last-known policy snapshot
Policy propagation	Normal changes visible within <code>10 s</code>	Versioned snapshot push plus periodic pull
Bucket state bound	Tokens within <code>[0, B]</code> at all times	Clamp inside the owner script; clamp again on capacity edits

Requirement	Target	Enforced by
Degraded-admission visibility	100% of emergency admissions tagged; alarm above 1% of decisions	Distinct outcome code on the decision and a dedicated metric

- Invariants
 - Tokens stay between 0 and capacity B.
 - Each atomic execution at the current bucket owner applies one debit.
 - Retried logical decisions need an atomic replay record to avoid a second debit.
 - With stable configuration and clock, admitted cost in an interval of duration t is at most $B + r \times t$.
 - This is a burst envelope, not a guarantee of exactly r requests in every one-second window.
 - Numerical targets above are interview assumptions unless a reference explicitly supplies them.
- Non-goals
 - Billing-grade metering.
 - An exact rolling-window count across regions.
 - Protecting the backend from long-running requests that have already been admitted.

Capacity and consequence — IC3 baseline

- Bucket state
 - 10,000 tenants × 5 route classes = 50,000 buckets.
 - At an assumed 200 B of logical state each, the raw state is 10 MB.
 - Allocator overhead and replication increase memory requirements.
 - Decision: memory is not the first constraint, so choose shard count from operations per second, not from state size.
- Decision traffic
 - One owner operation per request means 100,000 operations/s.
 - If a measured shard sustains 25,000 operations/s at the target latency, use at least 4 active shards before failure headroom.
 - This is a sizing example, not a Redis performance guarantee.
 - Decision: place replicas of each shard across fault domains and reserve enough failover capacity for the largest domain loss.
 - Adding an unrelated shard does not recover another shard's state or ownership.
- Hot tenant
 - A dominant tenant does not benefit from ordinary sharding.
 - Its bucket still has one serial owner.
 - Decision: monitor the hottest key and cap per-bucket rate before adding cluster nodes
 - The fix is disjoint grants, not more shards.

Architecture and flow — IC3 baseline

```

Data plane (per request):
Client -> edge abuse filter -> authentication -> gateway limiter -> backend
                                         |
                                         v
                                   bucket owner / Redis shard
                                   (primary + replica, different fault domains)

Control plane (per policy change):
Policy API -> durable policy DB -> versioned snapshots -> gateways
Decision outcomes -> bounded telemetry -> alarms and tenant reports

```

- Gateway
 - Derives tenant identity from authentication.

- Resolves a route template rather than an attacker-controlled raw URL.
 - Keeps a last-known validated policy snapshot, so the data plane keeps deciding when the control plane is down.
 - Cost: snapshot lag means gateways can apply different policy versions for a bounded window.
 - Owner
 - Uses a short atomic operation for refill and consume.
 - Redis scripting provides local execution atomicity.
 - Script execution occupies the shard, so avoid unbounded loops.
 - Cost: one serial owner per bucket caps the hottest tenant at one shard's throughput.
 - Policy API
 - A control-plane resource with `If-Match` versioning and an audit trail.
 - Cost: propagation is asynchronous, so an edit is not fleet-wide until every gateway acknowledges the version.
 - Backend
 - Enforces its own concurrency and queue limits.
 - A rate limiter alone cannot contain long-running requests.
 - Cost: two admission points must agree on which one owns overload rejection.
- Authenticate and derive `(tenant, route class)`.
- Read the active policy version from the local snapshot.
- Route the decision to the bucket owner.
- Atomically refill and conditionally consume.
- Allow the request or return `429` with a retry hint.
- On an owner timeout, apply the rule's explicit degradation policy.

API and data model — IC3 baseline

Surface	Contract	Concurrency or error behavior
<code>check(tenant, route, cost, request_id)</code>	Internal decision: allowed, remaining, retry delay, policy version	Validate $0 < cost \leq B$; do not retry an ambiguous consume blindly
<code>PUT /v1/tenants/{t}/limits/{route}</code>	Admin updates refill rate, burst, failure policy	<code>If-Match</code> rejects stale edits with <code>412</code>
<code>GET /v1/tenants/{t}/limits</code>	Paginated policy listing	Tenant authorization on every page
<code>POST /v1/tenants/{t}/limits/{route}:reset</code>	Privileged, audited quota reset	Idempotency key; a duplicate returns the first reset record
Rejected application request	<code>429</code> and <code>Retry-After</code> in seconds	Remaining tokens are an observation, not a reservation
Decision infrastructure unavailable	<code>503</code> for strict rules	Keep distinct from a quota rejection

State	Key	Fields and invariant
Durable policy	<code>(tenant_id, route_class)</code>	<code>rate</code> , <code>capacity</code> , <code>failure_policy</code> , <code>version</code> ; positive rate and capacity
Ephemeral bucket	<code>bucket:{tenant-route}</code>	<code>tokens</code> , <code>last_refill</code> , <code>applied_version</code> ; owner updates together
Optional decision replay	<code>bucket:{tenant-route}:req:{request_id}</code>	Prior decision; TTL equals the documented retry window
Audit event	<code>event_id</code>	Actor, old/new policy versions, timestamp; no credentials

- Reference DDL
 - Keys and uniqueness match the state table above and the [companion schema](#).
 - Bucket counters and decision replay entries live in Redis; only policy, reset, and audit rows are relational.

```

CREATE TABLE limit_policies (
  tenant_id text NOT NULL, route_class text NOT NULL,
  rate numeric NOT NULL CHECK (rate > 0), -- tokens per second
  capacity numeric NOT NULL CHECK (capacity > 0), -- burst B; check() validates 0 < cost <= B
  failure_policy text NOT NULL
  CHECK (failure_policy IN ('FAIL_OPEN_BOUNDED', 'FAIL_CLOSED')),
  fail_open_allowance numeric CHECK (fail_open_allowance >= 0), -- local allowance while the owner is
  unreachable
  version bigint NOT NULL CHECK (version > 0), -- If-Match validator; copied to the bucket
  as applied_version
  updated_at timestamptz NOT NULL,
  PRIMARY KEY (tenant_id, route_class),
  CHECK ((failure_policy = 'FAIL_CLOSED') = (fail_open_allowance IS NULL))
);
CREATE TABLE limit_resets (
  tenant_id text NOT NULL, route_class text NOT NULL, idempotency_key text NOT NULL,
  reset_id text NOT NULL, actor_id text NOT NULL, reason text NOT NULL,
  policy_version bigint NOT NULL, created_at timestamptz NOT NULL,
  PRIMARY KEY (tenant_id, route_class, idempotency_key), -- a duplicate returns the first reset
  record
  UNIQUE (tenant_id, reset_id)
); -- no FK: reset history outlives a deleted
policy
CREATE TABLE limit_audit_events (
  tenant_id text NOT NULL, event_id text NOT NULL, route_class text NOT NULL,
  actor_id text NOT NULL,
  action text NOT NULL CHECK (action IN ('POLICY_PUT', 'POLICY_DELETE', 'RESET')),
  old_version bigint, new_version bigint, -- versions only; no credentials or request
  bodies
  at timestamptz NOT NULL,
  PRIMARY KEY (tenant_id, event_id)
);
CREATE INDEX limit_audit_by_route ON limit_audit_events (tenant_id, route_class, at, event_id);

```

- Not captured by the DDL
 - `bucket:{tenant-route}` holds tokens, `last_refill`, `applied_version` in Redis under one owner operation; the SQL row is the source version it applies.
 - `bucket:{tenant-route}:req:{request_id}` is a Redis replay entry with the retry-window TTL; it never reaches SQL.
 - Paginated `GET /limits` walks the `(tenant_id, route_class)` primary key as its cursor; no extra index is needed.
- Deviation from the companion
 - Companion `scope` is `route_class`, `rate_per_second` is `rate`, `burst` is `capacity`.
 - Companion mode `ENFORCE/SHADOW` is replaced by `failure_policy`, which is the knob this file's requirements name; shadow mode is an IC4 rollout option, not a baseline column.
 - `limit_resets` and `limit_audit_events` are added so the reset endpoint's idempotency and the audit row in the state table have a home.
- Use a stable bucket key across policy edits.
 - A new version must not silently create a second full bucket.
 - On update, clamp existing tokens to the new capacity under the owner operation.
 - Refill up to the change boundary using the old rate when that boundary is available.
 - Document bounded transition behavior if gateways observe updates at different times.
- Idle expiration
 - Evict only after enough idle time for a full refill.
 - Recreating a bucket full is then consistent with its prior idle state.
 - Do not expire depleted state earlier just to save memory.

Atomic algorithm and races — IC3 baseline

```
atomic CHECK(bucket, policy, now, cost):
    require 0 < cost <= capacity and rate > 0
    reject invalid or permanently unsatisfiable requests without a retry estimate
    load tokens and last_refill, or initialize a full bucket
    elapsed = max(0, now - last_refill)
    tokens = min(capacity, tokens + elapsed * rate)
    last_refill = max(last_refill, now)
    if tokens < cost:
        store refreshed state
        return DENY, ceil((cost - tokens) / rate)
    tokens = tokens - cost
    store refreshed state
    return ALLOW, tokens
```

- Linearization point
 - The owner's conditional debit determines the decision.
 - `GET` followed by client-side `SET` loses updates under concurrency.
- Consistency boundary
 - Atomicity holds within one owner's script execution.
 - It does not extend across shards, across a failover, or across regions.
- Clock
 - Use a monotonic clock for an in-process bucket.
 - Use an owner-side time source for distributed decisions.
 - Clamp backward jumps without moving the stored time backward.
 - A forward jump can grant an early burst up to capacity.
 - Strict clock guarantees need an explicit clock and failover design.
- Same bucket example
 - `B=20`, `r=10/s`, and `tokens=0`.
 - After `0.5 s`, five unit-cost requests may pass.
 - A sixth concurrent request must observe the prior debits and fail.
- Ambiguous timeout
 - At `5 ms` the gateway stops waiting
 - The script may or may not have run.
 - A blind retry can debit twice
 - A blind allow can over-admit.
 - Baseline: apply the declared degraded policy after an ambiguous timeout.
 - Optional replay protection stores request ID, payload hash, debit, and result atomically with the bucket update.
 - Its TTL must cover the supported retry horizon and its durability follows the same failover limits.
- Multiple constraints
 - One script can atomically check and debit colocated route and tenant buckets.
 - Redis Cluster multi-key operations require a shared hash slot.
 - All script-accessed keys must be declared as keys to the script.
 - Independent shards cannot provide an all-or-nothing multi-bucket debit without coordination.
 - Rejecting after one conservative debit wastes budget
 - It does not inherently over-admit.

Failure, operations, and security

Failure	Blast radius	Detection	Degraded behavior	Recovery
Owner times out after consuming	One uncertain decision	Timeout counter	Strict rule returns 503 ; no blind debit retry	Optional short-lived decision-ID dedup for a required retry contract
Redis primary loses recent debits on failover	Buckets on that shard	Failover and admission metrics	Temporary extra budget is possible	Accept explicitly or use a stronger state owner
Owner shard unavailable	That shard's tenants	Error/latency alarms	Capacity rule uses bounded emergency budget; security rule rejects	Warm and validate state before normal admission
Policy DB or policy API unavailable	New policy edits only	Snapshot version lag	Gateways keep deciding on the last-known snapshot	Resume ordered snapshot delivery; reconcile versions
Gateway restarts with an empty snapshot	One gateway's traffic	Missing-policy counter	Serve a bundled default policy; never fail open by omission	Pull the current snapshot before joining the pool
Bad policy rollout	Targeted tenants	Shadow reject-rate delta	Keep last-known valid snapshot	Roll back policy and clamp bucket state
Owner clock jumps forward	One shard's buckets	Refill-rate anomaly	Early burst up to B per bucket, then normal	Clamp time source; alert on drift beyond the bound
Hot tenant saturates one shard	Co-located tenants on that shard	Hottest-key load and p99 by shard	Per-bucket rate cap sheds the hot tenant first	Move the hot key or issue disjoint grants
Backend concurrency saturation	One service	In-flight count and queue age	Reject admission even with available tokens	Reduce offered load and shed expensive operations

- Atomic execution is not durable global correctness.
 - Redis replication is normally asynchronous.
 - `WAIT` does not turn the deployment into a strongly consistent CP system.
 - See [Redis replication](#).
- Bounded emergency mode
 - A fixed gateway membership can divide a separate emergency allowance.
 - Autoscaling and duplicate membership can violate that sum.
 - If membership is uncertain, use admission leases or reject strict traffic.
 - Never describe unrestricted fail-open as backend protection.
- Observe
 - Decision latency by shard.
 - Allowed and rejected cost.
 - Emergency-mode admissions.
 - Hottest bucket load.
 - Policy version lag.
 - Page on sustained decision failures or backend overload
 - A high reject count alone may indicate correct enforcement.
- Retry discipline
 - Retry a timed-out check at most once, and only with a `request_id` the owner can deduplicate.
 - Otherwise classify the decision as degraded and apply the rule's policy.
 - Clients honor `Retry-After` with jitter
 - A synchronized retry storm is itself an overload.
- Tenant isolation
 - Bucket keys and policies are scoped by tenant
 - Route-class cardinality is bounded per tenant.

- Authorize tenant administration
- Audit resets and quota increases.
- Keep one tenant's buckets on one shard so a hot tenant's blast radius is that shard, not the fleet.
- Rollout and rollback safety
 - Roll out policy through shadow evaluation: compare proposed decisions without consuming the live bucket twice.
 - Canary a small tenant cohort
 - Roll back when legitimate rejection or backend saturation breaches the agreed threshold.
 - A new script version must read the previous bucket layout
 - Change layout under a new key prefix, not in place.

Alternatives and IC4 stretch

Option	What it is	Strength	Weakness	Choose when
Token bucket	Refilled budget	Explicit burst envelope	Not an exact rolling-window count	Default burst-tolerant API protection
Fixed window	Counter per interval	Small state	Boundary permits concentrated traffic	Coarse limits tolerate boundary bursts
Sliding log	Timestamp per admitted event	Exact window membership	State and pruning grow with traffic	Small strict attempt limits
Sliding counter	Weighted adjacent windows	Constant state	Error depends on traffic distribution	Approximation is accepted and measured
Leaky-bucket queue	Scheduled release	Smooth downstream work	Queueing increases latency	Work may wait rather than be rejected

- Base choice: token bucket.
 - The deciding variable is whether bursts are part of the contract.
- IC4 stretch — hot tenant
 - Allocate disjoint token grants to gateways.
 - Debit a grant centrally before any local spending.
 - Crashed gateways strand unused tokens.
 - Reclaim only after expiry plus the stated clock/lease safety bound.
 - Fresh grant delivery must not reset a still-valid prior grant.
- IC4 stretch — exact global budget
 - Split budget into non-overlapping regional allocations.
 - Rebalance asynchronously while conserving the total allocation.
 - A disconnected region can spend only its remaining allocation.
 - Strict shared bursts require coordination and may sacrifice availability.
- IC4 stretch — 10× load
 - At 1M checks/s, grants of 100 can reduce central allocation traffic toward 10,000/s under full utilization.
 - Measure wasted grants and skew before claiming a 100× improvement.
- IC4 stretch — cost and blast radius
 - The owner fleet is the dominant cost
 - Grants trade owner throughput for stranded budget on gateway loss.
 - Cell the owner fleet per tenant cohort so one shard's failover degrades a cohort, not the region.
- Weakest assumption
 - No single tenant exceeds one shard's measured ceiling.
 - Mitigation: alert on hottest-key load early and have the grant path ready before that tenant arrives.
- Over-engineering warning
 - Do not introduce cross-region consensus for a regional approximate quota.

Level signal — IC3 baseline vs IC4 stretch

Phase	IC3 must land	IC4 adds	Separating signal
Requirements	Burst allowed, regional scope, tenant-plus-route key, envelope invariant	Capacity rules versus security rules, each with a failure policy	Names the degradation contract before being asked
Estimates	Bucket state, operations per second, shard count	Memory is not the constraint; the hot key picks the scaling path	A number changes a decision
Architecture	Gateway, owner, policy plane with one decision trace	Snapshot lag and single-owner ceiling as the price of each split	Every component has a mechanism and a price
Deep dive	Atomic script, clock clamp, last-token race	Ambiguous timeout and the multi-bucket hash-slot constraint	Reasons about what atomicity does not cover
Operations	Failure table, emergency budget, shadow rollout	Gateways stay static-stable under policy-plane loss; cohort cells	Volunteers control-plane outage behavior
Wrap	Recap and burst-versus-exactness tradeoff	Single-owner ceiling named as the weakest assumption	Identifies the flaw before the interviewer does

Interview pushback and validation

- Q1: Why is a concurrent map insufficient?
 - **IC3 baseline**
 - Refill and debit are a compound update.
 - **IC4 stretch**
 - Local atomicity still does not cover failover or independent regions.
- Q2: Can retries exceed the quota?
 - **IC3 baseline**
 - An ambiguous debit retry can consume twice and reject legitimate work.
 - **IC4 stretch**
 - A lost replicated debit or duplicate token grant can instead over-admit.
- Q3: What happens when the entire limiter fails?
 - **IC3 baseline**
 - Select strict rejection or bounded emergency admission per policy.
 - **IC4 stretch**
 - Reserve capacity for recovery operations without bypassing tenant authorization.
- Q4: The requirement changes to exactly 100 attempts in any minute.
 - **IC3 baseline**
 - Use a sliding event log for this bounded attempt rate.
 - **IC4 stretch**
 - Coordinate the authoritative history across regions or explicitly reduce availability.
- Q5: Can we shard one tenant's key eight ways?
 - **IC3 baseline**
 - Independent full budgets multiply the allowance.
 - **IC4 stretch**
 - Disjoint allocations preserve a sum but trade utilization for availability.
- Q6: Why not a queue?
 - **IC3 baseline**
 - Rejection and delayed execution have different client contracts.
 - **IC4 stretch**
 - Queue bounds must include wait-time deadlines and cancellation cost.
- Q7 (premise reversal): Why a shared owner at all?
 - Put the limit in each gateway and divide the quota by fleet size.

- **IC3 baseline**
 - The sum is only correct with fixed membership
 - Autoscaling, uneven routing, and restarts break it silently.
- **IC4 stretch**
 - That is the grant model, and it is only sound when grants are leased, expire, and are reclaimed centrally.
- Q8: The policy database is down for an hour.
 - What do tenants see?
 - **IC3 baseline**
 - Decisions continue from the last-known snapshot
 - Only edits return 503.
 - **IC4 stretch**
 - Alarm on snapshot age and reconcile versions across the fleet before edits resume.
- Validation scenarios
 - Inject time to test refill boundaries and clock regression.
 - Race requests against the last token.
 - Change capacity while a bucket is depleted.
 - Lose a response after the owner debit.
 - Fail over with a lagging replica.
 - Restart a gateway holding a partially spent grant.
 - Stop the policy API and confirm decisions continue on the stale snapshot.

Recall card

- "I use an atomic token bucket keyed by authenticated tenant and route."
- "The normal invariant is a burst-plus-refill envelope."
- "Owner failure invokes an explicit strict or bounded-emergency policy."
- "The policy plane can die; gateways keep deciding on the last snapshot."
- "The first scaling issue is a hot bucket; disjoint grants change central traffic and recovery semantics."
- Wrap-up script
 - "The design is a regional token bucket per tenant and route, with one atomic owner per bucket and versioned policy pushed to gateways."
 - "The healthy owner enforces burst plus rate times elapsed time, while failover and timeout policies declare any additional admission budget."
 - "The main tradeoff is a burst envelope instead of an exact window, which buys one round trip per decision and no cross-region coordination."
 - "My weakest assumption is that no tenant exceeds one shard's ceiling; I would validate the hottest key first and keep the grant path ready."

References

- [Ranked question and report provenance.](#)
- [OCI role framing.](#)
- [Detailed API and schema companion.](#)
- Reference check: 2026-09-09.
 - Official documentation supports the mechanisms cited next to each link.
 - Capacity values and architecture choices remain proposed interview assumptions.
- [Redis atomic scripting](#): the local atomic execution boundary.
- [Redis replication](#): failover and durability limits.
- [Redis Cluster specification](#): hash slots and the multi-key constraint on scripts.